

MODELO DE DATOS UNIVERSAL PMS DEBACU

**Versión inicial – enfoque read-only, multi-property,
preparado para riesgo + revenue + analítica**

1. Objetivo del modelo

Definir un modelo de datos interno único para Debacu que permita normalizar información procedente de distintos PMS como:

- Tesipro / Ulyses
- Cloudbeds
- Mews
- apaleo
- Oracle OPERA
- otros PMS futuros

Este modelo debe permitir:

- integrar varios PMS sin rediseñar Debacu cada vez,
 - soportar organización con varias propiedades,
 - alimentar módulos de riesgo,
 - alimentar revenue intelligence,
 - soportar analítica histórica,
 - permitir sincronización incremental,
 - mantener trazabilidad del dato original.
-

2. Principios del modelo universal Debacu

2.1 Debacu manda, no el PMS

La estructura interna la define Debacu, no el proveedor PMS.

No vamos a crear tablas “tipo Mews”, “tipo Cloudbeds” o “tipo Tesipro”.
Cada PMS se adapta al modelo Debacu.

2.2 Separación entre dato canónico y dato bruto

Debacu debe guardar dos niveles:

A. dato normalizado/canónico

el que usa la aplicación, dashboards, métricas, riesgo y revenue.

B. dato bruto original

payload original del PMS, para:

- auditoría,
- depuración,
- trazabilidad,
- reimportación futura,
- ampliaciones de mapeo.

Esto es obligatorio. Si no lo guardas, luego no puedes corregir mapeos sin volver a pedir datos al PMS.

2.3 Multi-tenant real

Todo debe colgar de:

- org_id
- property_id

aunque el PMS no lo entregue bien.

Debacu necesita separar claramente:

- organización cliente,
 - propiedad hotelera,
 - proveedor PMS,
 - conexión PMS,
 - dato sincronizado.
-

2.4 Read-only en fase 1

El modelo inicial se diseña para lectura.

No hace falta meter desde el principio estructuras de escritura, comandos operativos ni workflows transaccionales sobre PMS.

2.5 Modelo preparado para cambios

Los PMS cambian nombres, estados, campos y formatos.

El modelo Debacu debe:

- normalizar lo esencial,
 - aceptar nulos donde haya diferencias,
 - mantener raw payload,
 - usar tablas de mapping y catálogos cuando haga falta.
-

3. Capas del modelo universal

Lo más sensato es organizarlo en 4 capas.

Capa 1 – Conexión e integración

Define de dónde viene el dato.

Capa 2 – Ingesta bruta

Guarda el payload original del PMS.

Capa 3 – Normalización canónica

Transforma el dato PMS al modelo universal Debacu.

Capa 4 – Consumo analítico

Usado por riesgo, revenue, dashboards, alertas y reporting.

4. Entidades maestras del modelo Debacu

Estas son las entidades base que deberías tener sí o sí.

4.1 PMS Provider

Catálogo de proveedores PMS.

Ejemplos:

- TESIPRO_ULYSES
- CLOUDBEDS
- MEWS
- APALEO
- OPERA
- SIHOT

Campos

```
PmsProvider {  
  code: string  
  name: string  
  active: boolean  
  auth_type: 'OAUTH2' | 'API_KEY' | 'CLIENT_CREDENTIALS' | 'CUSTOM'  
  supports_webhooks: boolean  
  supports_multi_property: boolean  
}
```

4.2 PMS Connection

Representa una conexión real entre Debacu y un PMS.

Puede ser:

- por organización
- por propiedad
- por cuenta PMS
- por grupo

Campos recomendados

```
PmsConnection {  
  id: uuid  
  org_id: uuid  
  property_id: uuid | null  
  provider_code: string  
  environment: 'sandbox' | 'production'  
  external_account_id: string | null  
  external_group_id: string | null  
  auth_mode: string  
  status: 'PENDING' | 'ACTIVE' | 'ERROR' | 'DISCONNECTED'  
  last_sync_at: timestampz | null  
  last_success_sync_at: timestampz | null  
  last_error_at: timestampz | null  
  last_error_message: text | null  
  created_at: timestampz  
  updated_at: timestampz  
}
```

Comentario

Aquí no guardes secretos en claro.

Las credenciales deben ir aparte, cifradas o en secrets manager.

4.3 PMS Property Mapping

Porque la propiedad en DebaCu y la propiedad en el PMS no siempre coinciden igual.

Campos

```
PmsPropertyMapping {
  id: uuid
  org_id: uuid
  property_id: uuid
  connection_id: uuid
  external_property_id: string
  external_property_code: string | null
  external_property_name: string | null
  is_primary: boolean
  created_at: timestampz
}
```

5. Tablas de ingesta bruta

Esto es imprescindible.

5.1 PMS Sync Job

Registro de cada sincronización.

Campos

```
PmsSyncJob {
  id: uuid
  org_id: uuid
  property_id: uuid | null
  connection_id: uuid
  provider_code: string
  entity_type: 'PROPERTY' | 'ROOM' | 'ROOM_TYPE' | 'GUEST' | 'RESERVATION' |
'STAY' | 'FOLIO'
  sync_mode: 'FULL' | 'INCREMENTAL'
  started_at: timestampz
  finished_at: timestampz | null
  status: 'RUNNING' | 'SUCCESS' | 'PARTIAL' | 'FAILED'
  records_read: integer
  records_inserted: integer
  records_updated: integer
  records_failed: integer
  error_summary: text | null
}
```

5.2 PMS Raw Record

Payload bruto por registro.

Campos

```
PmsRawRecord {
  id: uuid
  org_id: uuid
  property_id: uuid | null
  connection_id: uuid
  sync_job_id: uuid
  provider_code: string
  entity_type: 'PROPERTY' | 'ROOM' | 'ROOM_TYPE' | 'GUEST' | 'RESERVATION' |
'STAY' | 'FOLIO'
  external_id: string
  external_parent_id: string | null
  external_updated_at: timestampz | null
  payload_json: jsonb
  payload_hash: text
  first_seen_at: timestampz
  last_seen_at: timestampz
}
```

Comentario

payload_hash te ayuda a detectar cambios reales sin reprocesar siempre.

6. Modelo canónico Debacu

Aquí está lo importante de verdad.

6.1 Debacu Property

Normalmente ya la tienes, pero debe ser la referencia maestra.

Campos mínimos

```
DebacuProperty {
  id: uuid
  org_id: uuid
  code: text
  name: text
  status: text
  country_code: text | null
  city: text | null
  timezone: text | null
  currency_code: text | null
}
```

6.2 Room Type

Entidad universal para tipos de habitación.

Campos

```
PmsRoomType {  
  id: uuid  
  org_id: uuid  
  property_id: uuid  
  connection_id: uuid  
  external_room_type_id: string  
  code: text | null  
  name: text  
  description: text | null  
  capacity_adults: integer | null  
  capacity_children: integer | null  
  max_occupancy: integer | null  
  is_active: boolean  
  raw_status: text | null  
  created_at: timestampz  
  updated_at: timestampz  
}
```

Normalización

Da igual si un PMS lo llama:

- category
- roomType
- class
- accommodationType

internamente será PmsRoomType.

6.3 Room

Habitación física o unidad vendible.

Campos

```
PmsRoom {  
  id: uuid  
  org_id: uuid  
  property_id: uuid  
  connection_id: uuid  
  external_room_id: string  
  room_type_id: uuid | null  
  room_number: text | null  
  room_name: text | null  
  floor: text | null  
  building: text | null  
  is_active: boolean  
  operational_status: 'AVAILABLE' | 'OUT_OF_SERVICE' | 'DIRTY' | 'CLEAN' |  
  'UNKNOWN'  
  raw_status: text | null  
  created_at: timestampz  
  updated_at: timestampz  
}
```

Comentario

Muchos PMS no exponen housekeeping bien.

Por eso `operational_status` debe aceptar UNKNOWN.

6.4 Guest Profile

Perfil de huésped normalizado.

Campos

```
PmsGuest {
  id: uuid
  org_id: uuid
  property_id: uuid | null
  connection_id: uuid
  external_guest_id: string
  external_profile_id: string | null
  first_name: text | null
  last_name: text | null
  full_name: text | null
  email: text | null
  phone: text | null
  document_type: text | null
  document_number: text | null
  country_code: text | null
  nationality_code: text | null
  birth_date: date | null
  gender: text | null
  is_company: boolean | null
  company_name: text | null
  is_active: boolean
  created_at: timestampz
  updated_at: timestampz
}
```

Muy importante

No todos los PMS dan documento o fecha de nacimiento por API.

No fuerces esos campos como obligatorios.

6.5 Reservation

Es la entidad clave.

Campos

```
PmsReservation {
  id: uuid
  org_id: uuid
  property_id: uuid
  connection_id: uuid
  external_reservation_id: string
  external_confirmation_code: text | null
  external_group_reservation_id: text | null
  guest_id: uuid | null
}
```



```

primary_room_type_id: uuid | null
assigned_room_id: uuid | null
status: 'PENDING' | 'CONFIRMED' | 'CHECKED_IN' | 'CHECKED_OUT' | 'CANCELLED' |
'NO_SHOW' | 'IN_HOUSE' | 'UNKNOWN'
raw_status: text | null
booked_at: timestampz | null
check_in_date: date | null
check_out_date: date | null
nights: integer | null
adults: integer | null
children: integer | null
infants: integer | null
rooms_count: integer | null
channel_code: text | null
channel_name: text | null
segment_code: text | null
segment_name: text | null
rate_plan_code: text | null
rate_plan_name: text | null
currency_code: text | null
room_revenue_amount: numeric(14,2) | null
total_amount: numeric(14,2) | null
paid_amount: numeric(14,2) | null
balance_amount: numeric(14,2) | null
cancellation_at: timestampz | null
source_updated_at: timestampz | null
created_at: timestampz
updated_at: timestampz
}

```

Comentario clave

Aquí debes separar siempre:

- status normalizado
- raw_status original PMS

Porque cada PMS usa su propio vocabulario.

Ejemplo:

- one may say Confirmed
- another Booked
- another Reserved
- another Due In

Debacu traduce todo eso.

6.6 Reservation Guest Link

Porque una reserva puede tener varios huéspedes.

Campos

```
PmsReservationGuest {
```

```
id: uuid
reservation_id: uuid
guest_id: uuid
role: 'PRIMARY' | 'SECONDARY' | 'SHARER' | 'UNKNOWN'
is_main_guest: boolean
}
```

Esto te evita perder acompañantes.

6.7 Stay

Conviene separar reserva de estancia real.

¿Por qué?

Porque una reserva puede cambiar, cancelarse, dividirse, hacer check-in parcial o tener comportamiento operativo distinto.

Campos

```
PmsStay {
  id: uuid
  org_id: uuid
  property_id: uuid
  connection_id: uuid
  reservation_id: uuid | null
  guest_id: uuid | null
  room_id: uuid | null
  room_type_id: uuid | null
  external_stay_id: string | null
  stay_status: 'EXPECTED' | 'IN_HOUSE' | 'CHECKED_OUT' | 'CANCELLED' | 'NO_SHOW'
| 'UNKNOWN'
  raw_status: text | null
  arrival_scheduled_at: timestampz | null
  departure_scheduled_at: timestampz | null
  check_in_at: timestampz | null
  check_out_at: timestampz | null
  adults: integer | null
  children: integer | null
  currency_code: text | null
  room_revenue_amount: numeric(14,2) | null
  total_amount: numeric(14,2) | null
  created_at: timestampz
  updated_at: timestampz
}
```

Recomendación

Aunque algunos PMS no separen claramente reserva y estancia, Deba cu sí debería hacerlo.

6.8 Folio / Charges Summary

Solo si Revenue lo necesita.

Yo no empezaría con detalle contable fino, sino con resumen.

Campos

```
PmsFolioSummary {
  id: uuid
  org_id: uuid
  property_id: uuid
  connection_id: uuid
  reservation_id: uuid | null
  stay_id: uuid | null
  external_folio_id: string | null
  currency_code: text | null
  room_charges_amount: numeric(14,2) | null
  food_beverage_amount: numeric(14,2) | null
  other_charges_amount: numeric(14,2) | null
  taxes_amount: numeric(14,2) | null
  discounts_amount: numeric(14,2) | null
  total_amount: numeric(14,2) | null
  balance_amount: numeric(14,2) | null
  source_updated_at: timestampz | null
  created_at: timestampz
  updated_at: timestampz
}
```

7. Catálogos de normalización

Esto te va a salvar mucho trabajo.

No metas toda la lógica de equivalencias embebida en código.

Crea catálogos de mapping.

7.1 Reservation Status Mapping

```
PmsStatusMapping {
  id: uuid
  provider_code: string
  entity_type: 'RESERVATION' | 'STAY' | 'ROOM'
  raw_value: text
  canonical_value: text
  is_active: boolean
}
```

Ejemplo

- Reserved → CONFIRMED
 - Booked → CONFIRMED
 - InHouse → IN_HOUSE
 - Due Out → CHECKED_IN o IN_HOUSE según lógica
 - Cancelled → CANCELLED
-

7.2 Channel Mapping

```
PmsChannelMapping {
```

```
id: uuid
provider_code: string
raw_channel_code: text | null
raw_channel_name: text | null
canonical_channel: text
}
```

Ejemplo canónico

- DIRECT
 - BOOKING
 - EXPEDIA
 - AIRBNB
 - HOTELBEDS
 - WALK_IN
 - CORPORATE
 - UNKNOWN
-

7.3 Segment Mapping

Igual que canal.

Ejemplo canónico

- LEISURE
 - BUSINESS
 - OTA
 - GROUP
 - CORPORATE
 - WHOLESALE
 - DIRECT
 - UNKNOWN
-

8. Trazabilidad y claves externas

Esto también es crítico.

Cada entidad canónica que venga del PMS debe guardar:

- `connection_id`

- `external_*_id`
- `source_updated_at`
- `raw_status`
- relación con `PmsRawRecord` si quieres trazabilidad completa

Porque si mañana falla un mapping, tienes que poder reconstruir exactamente de dónde vino el dato.

9. Reglas de identidad

Aquí hay una decisión importante.

9.1 La identidad principal interna debe ser UUID DebaCu

Nunca uses como PK interna el ID del PMS.

El PMS ID cambia de un proveedor a otro y puede colisionar entre propiedades.

9.2 Clave única natural recomendada

Para cada entidad externa, usa unicidad compuesta:

Ejemplo reserva

`unique (connection_id, external_reservation_id)`

Ejemplo huésped

`unique (connection_id, external_guest_id)`

Ejemplo habitación

`unique (connection_id, external_room_id)`

Así soportas varios PMS y varias propiedades sin choques.

10. Modelo de sincronización incremental

No basta con tener tablas. Debes pensar el flujo.

10.1 Full sync inicial

Primera carga:

- propiedades
- room types

- rooms
 - huéspedes
 - reservas
 - stays
-

10.2 Sync incremental

Luego solo cambios desde:

- updated_at
- modified_since
- cursores
- webhooks
- fecha de última sync exitosa

según lo que permita cada PMS.

10.3 Soft deactivation

Si algo deja de venir del PMS, no siempre hay que borrarlo.

Ejemplo:

- habitación inactiva,
- room type despublicado,
- huésped no devuelto en incremental.

Mejor usar:

- is_active
 - last_seen_at
 - source_deleted_at si se detecta
-

11. Qué usarán los módulos Debacu

Ahora viene la parte útil de negocio.

11.1 Riesgo

Usará principalmente:

- huéspedes,
 - reservas,
 - stays,
 - property,
 - histórico de reservas/estancias,
 - país, documento, canal, comportamiento temporal.
-

11.2 Revenue

Usará:

- reservas,
 - stays,
 - room types,
 - rooms,
 - rate plans,
 - canales,
 - segmentos,
 - importes.
-

11.3 Analítica multi-propiedad

Usará:

- org_id
 - property_id
 - reservation status normalizado
 - stay status normalizado
 - canal normalizado
 - segmento normalizado
 - room type normalizado
-

12. Qué no meter aún en el modelo v1

Para no sobrediseñar, yo dejaría fuera por ahora:

- housekeeping detallado,
- maintenance tickets,
- POS detallado,
- pagos transaccionales,
- tarjetas tokenizadas,
- pricing rules,
- inventory writeback,
- mensajes internos PMS,
- tareas operativas.

Todo eso puede llegar luego, pero no lo metas en v1.

13. Estructura recomendada en Supabase/Postgres

Yo lo organizaría así, en schemas o al menos en naming lógico.

Integración

- pms_providers
- pms_connections
- pms_property_mappings
- pms_sync_jobs
- pms_raw_records

Modelo canónico

- pms_room_types
- pms_rooms
- pms_guests
- pms_reservations
- pms_reservation_guests

- pms_stays
- pms_folio_summaries

Catálogos / mappings

- pms_status_mappings
 - pms_channel_mappings
 - pms_segment_mappings
-

14. Ejemplo práctico de conversión

Caso Cloudbeds / Mews / Tesipro

Supón que cada PMS devuelve algo distinto:

Cloudbeds

```
{
  "reservationID": "CB123",
  "status": "confirmed",
  "guestName": "Juan Perez",
  "checkIn": "2026-03-10",
  "checkOut": "2026-03-12"
}
```

Mews

```
{
  "Id": "MW789",
  "State": "Started",
  "Customer": {
    "FirstName": "Juan",
    "LastName": "Perez"
  },
  "Arrival": "2026-03-10T15:00:00Z",
  "Departure": "2026-03-12T12:00:00Z"
}
```

Tesipro/Ulyses

```
{
  "booking_id": "TS456",
  "booking_status": "inhouse",
  "client_name": "Juan Perez",
  "arrival_date": "2026-03-10",
  "departure_date": "2026-03-12"
}
```

Debacu lo convierte a algo así

```
{
  "external_reservation_id": "...",
  "status": "CHECKED_IN",
}
```

```
"raw_status": "...",  
"full_name": "Juan Perez",  
"check_in_date": "2026-03-10",  
"check_out_date": "2026-03-12"  
}
```

Eso es exactamente lo que necesitas:

que el dashboard de Debacu no tenga que saber si viene de Mews, Tesipro o Cloudbeds.

15. Reglas de diseño para Claude Code

Esto es lo que deberías meter como criterio fijo para Claude.

Instrucciones de arquitectura

Claude debe asumir siempre que:

1. Debacu usa un modelo canónico universal PMS.
 2. Ningún conector PMS puede imponer su estructura al core de Debacu.
 3. Todo dato PMS debe almacenar:
 - conexión,
 - propiedad,
 - external id,
 - raw payload,
 - timestamps de origen.
 4. Todo estado PMS debe normalizarse a catálogos Debacu.
 5. Las integraciones v1 son read-only.
 6. El modelo debe soportar multi-property dentro de una misma organización.
 7. El frontend nunca hablará directamente con el PMS.
 8. Toda lógica de normalización vive en backend/integration layer.
 9. Debacu debe poder añadir nuevos PMS sin romper consultas existentes.
 10. Los conectores deben ser idempotentes.
-

16. Conclusión realista

El error típico aquí es intentar integrar PMS por “pantallas” o por “casos rápidos”.

Eso funciona dos semanas. Luego se vuelve inmantenible.

Lo correcto para Debacu es:

- diseñar **modelo universal**,
- guardar **raw + canonical**,
- normalizar estados y catálogos,
- separar **reserva** de **estancia**,
- soportar **org + property + connection**,
- construir conectores como adaptadores.